

13. Bölüm

Tip Dönüşümleri ve Göstericiler

13.1 Tip Dönüşümleri

C# dilinde derleme öncesi bir değişken bildirildikten sonra, bu tip örtük olarak bir başka değişkenin tipine dönüştürülebilir olmadığı sürece tekrar bildirilemez veya başka bir tipte bir değer atanamaz. Örneğin, `string` örtük olarak `int` veri tipine dönüştürülemez. Bu nedenle, bir değişken `int` olarak bildirdikten sonra, aşağıdaki koddaki gibi, ona "11" metnini atayamazsınız;

```
int i;  
i = "11"; // error CS0029: can't implicitly convert type 'string' to 'int'
```

Ancak bazen bir değeri başka bir tipin değişkenine veya yöntem parametresine kopyalamamız gerekebilir. Örneğin, `double` parametresi bekleyen bir yöntemle geçirmemiz gereken bir tam sayı değişkenimiz olabilir. Bir başka örnek olarak bir sınıf örneğini (`instance`) bir arayüz (`interface`) tipinin değişkenine atamız gerekebilir. Bu tür işlemlere tip dönüşümleri denir;

- ♦ **Örtük dönüşümler (implicit conversion):** Dönüştürme her zaman başarılı olduğundan ve veri kaybı olmadığından özel bir söz dizimi gerekmez. Örnek olarak küçükten daha büyük tam sayı türlerine dönüştürmeler ve türetilmiş sınıflardan temel sınıflara dönüştürmeler verilebilir.
- ♦ **Açık dönüşümler (explicit conversion) veya zorlama (casting):** Açık dönüşümler bir atama ifadesi gerektirir. Dönüşümde bilgi kaybolabileceği veya dönüştürmenin başka nedenlerle başarılı olmaya-bileceği durumlarda atama gereklidir. Buna örnek olarak da daha az duyarlığa veya daha küçük bir belleğe sahip bir tipe sayısal dönüştürme ve temel sınıf örneğinin türetilmiş bir sınıfa dönüştürülmesi verilebilir.
- ♦ **Kullanıcı tanımlı dönüşümler:** Kullanıcı tanımlı dönüştürmeler, temel sınıf türetilmiş sınıf ilişkisi olmayan özel türler arasında açık ve örtük dönüştürmeleri etkinleştirmek için tanımlayabileceğiniz özel yöntemler kullanır.
- ♦ **Yardımcı sınıflarla dönüşümler:** Uyumlu olmayan tipler arasında dönüştürme yapmak için yardımcı sınıflar kullanılır. `System.Convert`, `Parse`, `Int32.Parse` ve `System.BitConverter` bu yardımcı sınıf ve yöntemleridir.

13.2 Örtük Dönüşümler

Derleyicinin herhangi bir özel müdahale gerektirmeden, verinin güvenli bir şekilde taşınabileceğini garanti ettiği durumlarda **örtük dönüşüm (implicit conversion)** uygulanır. Genellikle küçük (bellekte az yer kaplayan) bir veri türü, kendisinden daha geniş bir türe aktarılırken kullanılır. Veri kaybı ihtimali yoktur. Hedef veri tipin kapasitesi, kaynak veri tipinden büyük olmalıdır.

```
int tamSayi = 123;  
double ondalikliSayi = tamSayi; // Implicit: int, double'ın içine rahatça sığar.
```

```
char karakter = 'A';  
int asciiKarsiligi = karakter; // Implicit: char 16-bit, int 32-bit'tir.
```

Referans tipler için, örtük dönüştürme her zaman bir sınıftan doğrudan veya dolaylı temel sınıflarından veya arayüzlerinden herhangi birine var olur. Türetilmiş bir sınıf her zaman bir temel sınıfın tüm üyelerini içerdiğinden özel söz dizimi gerekmez.

```
Derived d = new Derived();  
Base b = d; // Türetilmiş sınıf, her zaman taban sınıfa örtük olarak dönüştürülür.
```

13.3 Açık Dönüşümler

Veri kaybı riski olan veya dönüşümün her zaman başarılı olamayacağı durumlarda **açık dönüşümler (explicit conversion)** yapılır. Derleyiciye "Ne yaptığımı biliyorum, sorumluluk bende" demek için parantez içinde hedef tür belirtilerek dönüşüme zorlanır (`casting`). Veri kaybı veya taşma yaşanabilir. Bundan dolayı risk yüksektir. Büyük (bellekte çok yer kaplayan) veri tipinden küçük veri tipine geçerken veya uyumsuz türler arasında kullanılır. Bu dönüşümde bir atama gerçekleştirmek için, dönüştürülecek değerin veya değişkenin

önündeki parantez içinde atama yaptığınız veri tipi belirtilerek **zorlama işleci** (**cast operator**) kullanılır;

```
double pi = 3.14;
int tamPi = (int)pi; // Explicit: Ondalık kısım (.14) atılır. Veri kaybı!

long büyükSayi = 9876543210;
int küçükSayi = (int)büyükSayi; // Explicit: Veri taşması yaşanabilir! int veri tipine zorlandı
→ (cast)
```

Bu program, **zorlama** (**cast**) olmadan derlenmez. Referans tipler için, bir taban tipten türetilmiş tipe dönüştürmeniz gerekiyorsa zorlama yine gereklidir;

```
Derived d = new Derived(); // Türetilmiş tipten bir nesne imal ediliyor.
Base b = d; // Örtük (implicit) olarak taban tipe dönüştürülebilir.
/*
Ancak bu taban tipteki b nesnesi türetilmiş tipe dönüştürülecek ise açık olarak dönüştürülmesi
→ gerekir.
*/
Derived d2 = (Derived)b;
```

Referans tipleri arasındaki dönüştürme işlemi, temel alınan nesnenin çalışma zamanı (**run-time**) tipini değiştirmez. Yalnızca bu nesneye referans olarak kullanılan değerin tipini değiştirir. Yani çok biçimlilik (**polymorphism**) burada uygulanır. Bazı referans tipi dönüştürmelerinde, derleyici bir zorlamanın geçerli olup olmadığını belirleyemez. Doğru derlenen bir açık dönüşüm işleminin çalışma zamanında başarısız olması mümkündür;

```
Canlı canlı = new Hayvan();
Bitki bitki = (Bitki)canlı; // Derlenir ama çalıştırma anında hata olur:InvalidCastException
```

```
class Canlı
{
    public virtual string ToString() => "Ben Bir Canlıyım.";
}
class Hayvan : Canlı {
    public void HareketEt() => System.Console.WriteLine("Hareket Ediyorum..");
}
class Bitki : Canlı {
    public void FotosentezYap() => System.Console.WriteLine("Fotosentez Yapıyorum..");
}
```

Örtük ve açık dönüşümler aşağıdaki tabloda özetlenmiştir;

Özellik	Örtük	Açık
Güvenlik	Tamamen Güvenli	Riskli (Veri kaybı/Taşma)
Sözdizimi	Doğrudan atama (a = b)	Zorlama işleci (cast operator) (((int)b))
Yön	Genişleten (widening)	Daraltan (narrowing)
Derleyici	Otomatik yapar	Müdahale bekler

Tablo 13.1: Örtük ve Açık Dönüşüm Karşılaştırması

Açık dönüşüm sırasında bir **long** değeri **int** kapasitesini aşarsa ne olur? C# derleyicisi bizi uyarır mı? Varsayılan olarak C# taşmaları görmezden gelir (**silent overflow**) ve yanlış bir sonuç üretir. Ancak bu dönüşümü bir **checked** bloğu içine alırsanız, taşma yaşandığında program **OverflowException** fırlatır. Özel olarak bu kontrol yapılmasını istiyorsanız **unchecked** bloğu yazarsınız.

```
long uzunTamsayı=0x11FFFFFFFFFFFFFFF;
int sonuç;
sonuç = (int)uzunTamsayı; // Limit aşılrırsa hata fırlatmaz!
Console.WriteLine($"{sonuç:X}"); //FFFFFFF

unchecked {
    sonuç = (int)uzunTamsayı; // Limit aşılrırsa hata fırlatmaz!
    Console.WriteLine($"{sonuç:X}"); //FFFFFFF
}

checked {
    sonuç = (int)uzunTamsayı; /*Limit aşılrırsa hata fırlatır! Unhandled exception.
→ System.OverflowException: Arithmetic operation resulted in an overflow. */
```

```
Console.WriteLine($"{sonuç:X}");
}
```

13.4 Kullanıcı Tanımlı Dönüşümler

Kullanıcı tanımlı bir veri tipi, aynı iki tip arasında standart bir dönüşüm olmadığı sürece, başka bir tipe özel olarak **örtük** (**implicit**) veya **açık** (**explicit**) dönüşüm tanımlayabilir. Örtük dönüşümler, çağrılması için özel bir söz dizimi gerektirmez. Bu dönüşümler atamalarda ve yöntem çağrılarında meydana gelebilir. Önceden tanımlanmış örtük dönüşümleri her zaman başarılı olur ve asla bir istisna oluşturmaz. Kullanıcı tanımlı örtük dönüşümler de bu şekilde davranmalıdır.

Özel bir dönüşüm bir istisna oluşturabilir veya bilgi kaybedebilirse, bunu açık bir dönüşüm olarak tanımlamamız gerekir. Nesnelerle ilgili olan 8.20 başlığındaki **is** ve **as** işleçleri kullanıcı tanımlı dönüşümleri dikkate almaz. Kullanıcı tanımlı açık bir dönüşümü çağırarak için bir dönüştürme ifadesi kullanılması gerekir.

Sırasıyla örtük veya açık bir dönüşümü tanımlamak için **implicit operator** veya **explicit operator** anahtar sözcükleri kullanılır. İşleçlerin **static** tanımlandığı daha önce anlatılmıştı. Bir dönüşümü tanımlayan tip, o dönüşümün kaynak tipi veya hedef tipi olmalıdır. İki kullanıcı tanımlı tip arasındaki bir dönüşüm, iki tipten herhangi birinde tanımlanabilir.

```
// Implicit Kullanımı: Parantez içinde veri tipi belirtmeye gerek yok.
double doubleHız = 300.0;
Hız hızNesnesi = doubleHız;

int tamSayıHız = (int)hızNesnesi; // Explicit Kullanımı: (int) yazmak zorunludur.

Console.WriteLine($"Hız Nesnesi: {hızNesnesi.MetreSaniye} m/s"); //Hız Nesnesi: 300 m/s
Console.WriteLine($"Tam Sayı Hız: {tamSayıHız}"); //Tam Sayı Hız: 300

public struct Hız
{
    public double MetreSaniye { get; set; }

    // Implicit: double'dan Hız'a geçerken casting gerekmesin
    public static implicit operator Hız(double d) => new Hız { MetreSaniye = d };

    // Explicit: Hız'dan int'e geçerken hassasiyet kaybolacağı için zorunlu kılalım
    public static explicit operator int(Hız h) => (int)h.MetreSaniye;
}
```

13.5 Yardımcı Sınıflarla Dönüşümler

Çoğu programımızda klavyeden girilen metni bir metin (**string**) veri tipinde değişkenlerle işleriz. Bazı durumlarda da bu dizgileri değer tiplere dönüştürürüz. Bunun için **ToString()**, **System.Convert()** ve **TryParse()** yöntemlerini kullanırız.

```
using System.Globalization;

DateTime zaman = new(1971,9,1,11,53,0);
Console.WriteLine(zaman.ToString("yyyy-MM-dd HH:mm")); // 1971-09-01 11:53
Console.WriteLine(zaman.ToString(CultureInfo.InvariantCulture)); // 09/01/1971 11:53:00
Console.WriteLine(zaman.ToString(new CultureInfo("en-us"))); // 9/1/1971 11:53:00 AM
Console.WriteLine(zaman.ToString(new CultureInfo("tr-TR"))); // 1.09.1971 11:53:00
Console.WriteLine(zaman.ToString(new CultureInfo("ja-JP"))); // 1971/09/ 01 11:53:00

string doubleMetin = "12,53";
double sayi = Convert.ToDouble(doubleMetin); //YARDIMCI SINIF ile Dönüşüm
Console.WriteLine(sayi); // 12,53
string intMetin = "6161";
int num = int.Parse(intMetin);
Console.WriteLine(num); // 6161
if (int.TryParse(intMetin, out int result)) //YARDIMCI SINIF ile Dönüşüm
{
    Console.WriteLine(result); // 6161
}
```

```
else
{
    Console.WriteLine("Dönüştürme Başarısız Oldu.");
}
```

13.6 Kutulama ve Kutudan Çıkarma

§13.6.1 Kutulama

Kutulama (**boxing**), bir değer tipinin (**int**, **struct**, **double**, ...) bir referans tipine (**object** veya arayüz) dönüştürülmesi işlemidir. Bu işlem sırasında:

- ◊ Öbek (**heap**) bölgesinde yeni bir nesne tahsis edilir.
- ◊ Yığın(**stack**) bölgesindeki değer, Öbek (**heap**) bölgesindeki bu yeni nesnenin içine kopyalanır.
- ◊ Yığın (**stack**) bölgesinde bu nesnenin adresini tutan bir referans oluşturulur.

```
int sayi = 123;           // Değer tipi (Stack)
object kutu = sayi;       // BOXING (Heap'e taşındı)
```

§13.6.2 Kutudan Çıkarma

Kutudan çıkarma (**unboxing**), öbek(**heap**) bölgesinde kutulanmış olan değer tekrar kendi orijinal değer tipine dönüştürülmesidir. Bu işlem açıkça (**explicit**) bir tür dönüşümü olan zorlama (**cast**) gerektirir.

```
object kutu = 123;        // Boxing
int sayi = (int)kutu;      // UNBOXING (Heap'ten Stack'e geri kopyalandı)
```

Kutudan çıkarma (**unboxing**) yaparken tipin birebir uyuşması gerekir. Eğer **int** olarak kutulanan bir değeri **double** olarak çıkarmaya çalışırsanız **InvalidCastException** hatası alırsınız. Ayrıca performans etkisi de göz önüne alınmalıdır;

- ◊ **Zaman Maliyeti:** Kutulama (**boxing**) işlemi, yeni bir nesne tahsis etme ve veri kopyalama gerektirdiği için doğrudan atamalardan çok daha yavaştır.
- ◊ **Bellek Maliyeti:** Öbek (**heap**) bölgesinde sürekli yeni nesneler oluşmasına neden olur. Bu da **çöp toplayıcı** (GC) üzerindeki baskıyı artırır ve uygulamanın çöp toplayıcıdan dolayı ara ara "donmasına" yol açabilir.

Örnek olarak aşağıdaki kodu inceleyelim.

```
int yaş = 25;
Console.WriteLine("Yaşınız: " + yaş);
```

Yukarıda yazılan kodda kaç tane kutulama yapılmıştır? Burada gizli bir kutulama (**boxing**) vardır! Konsola bir şey yazmak için kullanılan **Console.WriteLine** veya **string.Concat** gibi yöntemler, sayısal bir değeri **string** ile birleştirirken onu **object** olarak ele alır. Bu yüzden yoğun döngülerde **yaş.ToString()** kullanmak, kutulama işlemini engelleyeceği için daha performanslıdır.

13.7 Göstericiler

C# dili, göstericileri sınırlı bir ölçüde destekler. Gösterici (**pointer**), bir bellek adresini tutan bir değişkenden başka bir şey değildir. C# göstericisi yalnızca değer tiplerin ve dizilerin bellek adresini tutmak için bildirilebilir (**declaration**).

1990 yıllarının sonlarına kadar, geliştirme çalışmalarımın çoğu C programlama dilini kullanarak yapıldı. C veya C++ dillerini kullanan programcılar göstericileri kullanmaktan kaçınmazdı. C# ve Java gibi saf nesne yönelimli dilleri kullanmaya başladığımızdan beri göstericiler çok nadir kullanılmaya başlandı ve genellikle buna gerek duyulmamaktadır. Çünkü göstericilerin çokça kullanıldığı dinamik veri yapılarını sağlayan kütüphaneler doğrudan bize bu yeni altyapılar tarafından sunulmuştur. Ancak göstericiler, bazı durumlarda düşük düzey iş ve işlemler için gerekli olabilir. Buna örnek olarak **P/Invoke** verilebilir. **P/Invoke**, yönetilen kodunuzdan (**managed code**) yönetilmeyen kütüphanelerdeki (**unmanaged code**) yapılara, geri aramalara (**callback**) ve fonksiyonlara erişmenizi sağlayan bir teknolojidir.

Referans tiplerin aksine, gösterici tipleri varsayılan çöp toplama (GC) mekanizması tarafından izlenmez. Aynı nedenden dolayı göstericilerin bir referans tipe veya hatta bir referans tipi içeren bir yapı (**struct**) türüne işaret etmesine izin verilmez. Göstericilerin yalnızca yönetilmeyen (**unmanaged**) tipleri, yani tüm temel veri tiplerini (**sbyte**, **byte**, **short**, **ushort**, **int**, **uint**, **long**, **ulong**, **char**, **float**, **double**, **decimal**, **bool**), **enum** tiplerini, diğer gösterici tiplerini ve yalnızca yönetilmeyen tipleri içeren yapıları (**struct**) işaret edebileceğini söyleyebiliriz.

Göstericiler başlığındaki verilen örneklerin çalışabilmesi için proje dosyalarında (.csproj) `<AllowUnsafeBlocks>true</AllowUnsafeBlocks>` ayarı yapılmalıdır!

§13.7.1 Gösterici Tipi Bildirme

Bir gösterici tipi bildiriminin genel biçimi aşağıda gösterildiği gibidir:

```
veritipi *gösterici-kimliği;
```

Yıldız (*) karakteri, **referanstan çıkarma işleci** (**de-reference operator**) olarak bilinir. Bir `int` tipinin adresini tutabilen bir gösterici değişkeni `ptrToInt` aşağıdaki gibi bildirir;

```
int* ptrToInt;
```

Referans işleci (&) bir değişkenin bellek adresini almak için kullanılabilir. Bir göstericiye ilk değer vermek için kullanılabilir. Aşağıdaki örnekte gösterici veri tipiyle, değişkenin veri tipinin aynı olduğuna dikkat edilmelidir.

```
int x = 100;
int* ptrToInt = & x;
Console.WriteLine((int) ptrToInt) // Konsola ptrToInt göstericisinin tuttuğu adres yazılır
Console.WriteLine(*ptr) // Konsola ptrToInt göstericisinin gösterdiği adresteki değer yazılır.
```

C# ifadeleri güvenli (**safe**) veya güvenli olmayan (**unsafe**) bir bağlamda yürütülebilir. **unsafe** anahtar sözcüğü kullanılarak güvenli olmayan olarak işaretlenen ifadeler çöp toplayıcısının (GC) denetimi dışında çalışır. C# dilinde göstericiler içeren herhangi bir kodun güvenli olmayan bir bağlam gerektirdiğini unutulmamalıdır. **Main** dahil tüm yöntemleri güvenli olmayan olarak işaretleyebilir kullanabiliriz.

```
unsafe
{
    int x = 1;
    int y = 2;
    int* ptr1 = &x;
    int* ptr2 = &y;
    Console.WriteLine((int)ptr1); //871885156
    Console.WriteLine((int)ptr2); //871885152
    Console.WriteLine(*ptr1); //1
    Console.WriteLine(*ptr2); //2
}
Sınıf nesne = new Sınıf();
nesne.GüvenliOmlayanYöntem();
class Sınıf
{
    public unsafe void GüvenliOmlayanYöntem()
    {
        int x = 3;
        int y = 4;
        int* ptr1 = &x;
        int* ptr2 = &y;
        Console.WriteLine((int)ptr1); //871885052
        Console.WriteLine((int)ptr2); //871885048
        Console.WriteLine(*ptr1); //3
        Console.WriteLine(*ptr2); //4
    }
}
```

C# çöp toplayıcısı (GC), çöp toplama işlemi sırasında nesneleri istediği gibi bellekte taşıyabilir. Çöp toplayıcısına bir nesneyi taşımasını söylemek için özel bir **fixed** anahtar sözcüğü kullanılır. Böylece işaret edilen değer türlerinin konumunu bellekte sabitletir. Bu durum **sabitleme** (**pinning**) olarak bilinir. Aşağıda C dili göstericileri gibi bir dizi elemanlarına erişim ve nesne durumuna erişim örneği verilmiştir;

```
unsafe
{
    var buffer = new int[1024];
    fixed (int* p = &buffer[0])
    {
```

```

        for (var i = 0; i < buffer.Length; i++)
        {
            *(p + i) = i;
        }
    }

    Sınıf nesne = new Sınıf();
    nesne.alan = 32;
    nesne.KonsolaAlanıYaz();
    fixed (int* alan = &nesne.alan)
    {
        *alan += 3;
    }
    nesne.KonsolaAlanıYaz();
}

```

```

class Sınıf
{
    public Sınıf() { }
    public int alan; //field
    public void KonsolaAlanıYaz()
    {
        Console.WriteLine("alan = {0}", alan);
    }
}

```

C#'ta `stackalloc` işleci, belleği yığın (**stack**) üzerinde tahsis eder ve bu işlemin sonucunda size o bellek bloğunun başlangıç adresini döndürür. Bu adresi yönetmek için en temel yol bir gösterici kullanmaktır. Normal bir dizi (`new int[]`) kullandığınızda, dizi nesnesi öbek (**heap**) üzerinde oluşur ve **çöp toplayıcı** (GC) tarafından takip edilir. Ancak `stackalloc` ile bir gösterici kullandığınızda bellek tahsisi ve serbest bırakılması neredeyse maliyetsizdir. Bellek yığında olduğu için çöp toplayıcının haberi bile olmaz, bu da "**stop-the-world**" duraksamalarını engeller. İşlemci, yığın belleğine çok daha hızlı erişir.

Göstericilerle çalışmak "güvensiz" kabul edildiği için bu kod bloğunun `unsafe` anahtar kelimesi ile işaretlenmesi gerekir.

// Proje ayarlarından "Allow unsafe code" seçeneği işaretlenmelidir.

```

unsafe
{
    // 1. Yığında 5 adet tam sayılık (20 byte) yer aç ve adresini 'p' göstericisine ata
    int* p = stackalloc int[5];

    // 2. Gösterici aritmetiği veya indisleme ile değer atama
    for (int i = 0; i < 5; i++)
    {
        // p[i] kullanımı aslında *(p + i) ile aynıdır
        p[i] = (i + 1) * 10;
    }

    // 3. Veriye erişim
    Console.WriteLine($"İlk eleman (p[0]): {*p}"); // Dereferencing
    Console.WriteLine($"Üçüncü eleman (p[2]): {*(p + 2)}"); // Adres aritmetiği

    // Bellek, bu 'unsafe' bloğun veya metodun dışına çıkıldığında otomatik geri alınır.
}

```

`stackalloc` ile ayrılan ve gösterici ile tutulan bellek, sadece tanımlandığı yöntemin yaşam süresi boyunca geçerlidir. Bu göstericiyi bir sınıfın özelliği (**property**) olarak saklamaya veya metod dışına döndürmeye çalışmak hatalı/tehlikelidir. Göstericilerde sınır kontrolü yoktur. Eğer 5 birimlik yer ayırıp `p[10]` dersanız, komşu bellek alanlarını bozabilir (**buffer overflow**) veya uygulamanın çökmesine neden olabilirsiniz. Eğer mutlak bir performans zorunluluğu yoksa, C#7.2+ ile gelen `Span<T>` yapısı, `stackalloc` kullanımını `unsafe` bloğuna ihtiyaç duymadan ve sınır kontrolleri (**bounds check**) yaparak daha güvenli hale getirir.

`stackalloc` Ne Zaman Kullanılmalı?

- ◇ **Küçük Veri Grupları:** Yığın boyutu sınırlıdır (genellikle 1 MB). Çok büyük dizileri yığında oluşturmaya çalışmak `StackOverflowException` hatasına neden olur.
- ◇ **Kısa Ömürlü Diziler:** Eğer dizi sadece o yöntemin içinde kullanılıp atılacaksa `stackalloc` idealdir.
- ◇ **Performans Kritik İşler:** Saniyede binlerce kez çalışan döngüler içinde sürekli yeni dizi oluşturmanız gerekiyorsa, çöp toplayıcı (GC) baskısını önlemek için kullanılır.

§13.7.2 Gösterici Aritmetiği

Göstericilerde toplama ve çıkarma tam sayılardan farklı şekilde çalışır. Bir gösterici artırıldığında veya azaltıldığında, işaret ettiği adres, referans tipinin bellek boyutu kadar artırılır veya azaltılır. Örneğin, `int` tipi 4 bayt boyutundadır. Bir `int`, 0 adresinde saklanıyorsa, bir sonraki `int`, 4 adresinde saklanır;

unsafe

```
{
    int* ptr = (int*)IntPtr.Zero;
    Console.WriteLine(new IntPtr(ptr)); // 0
    ptr++;
    Console.WriteLine(new IntPtr(ptr)); // 4
    ptr++;
    Console.WriteLine(new IntPtr(ptr)); // 8
}
```

Benzer şekilde, `long` tipi 8 bayt boyutundadır. Eğer bir `long`, 0 adresinde saklanıyorsa, bir sonraki `long` 8 adresinde saklanır;

unsafe

```
{
    long* ptr = (long*)IntPtr.Zero;
    Console.WriteLine(new IntPtr(ptr)); // 0
    ptr++; // bellekteki bir sonraki long adresini alır
    Console.WriteLine(new IntPtr(ptr)); // 8
    ptr+=4; // bellekteki 4 sonraki long adresini alır
    Console.WriteLine(new IntPtr(ptr)); // 40
}
```

Hiçbir değeri olmayan veri tipi anlamındaki `void` özeldir ve `void` göstericileri de özeldir. Veri tipi bilinmediğinde veya önemli olmadığında her şeyi ifade eden `void` göstericileri kullanılırlar. C ve C++ dilinin aksine boyuttan bağımsız yapıları nedeniyle, `void*` göstericileri artırılmaz veya azaltılmaz:

unsafe

```
{
    void* ptr = (void*)IntPtr.Zero;
    Console.WriteLine(new IntPtr(ptr));
    ptr++; // Hata: The operation in question is undefined on void pointers
    Console.WriteLine(new IntPtr(ptr));
    ptr++; // Hata: The operation in question is undefined on void pointers
    Console.WriteLine(new IntPtr(ptr));
}
```

Herhangi bir gösterici tipi, örtük (**implicit**) bir dönüşüm kullanılarak `void*` tipine atanabilir;

```
int* p1 = (int*)IntPtr.Zero;
void* ptr = p1; //örtük olarak void* tipine atama.
Bunun tersi durum ancak açık (explicit) olarak mümkündür;
int* p1 = (int*)IntPtr.Zero;
void* ptr = p1; //örtük olarak void* tipine atama.
int* p2 = (int*)ptr; //void* tipi int* tipine örtük olarak atanamaz!
```

§13.7.3 Referanstan Çıkarma İşleci

C ve C++ dillerinde, bir gösterici değişkeninin bildirimindeki yıldız karakteri, bildirilen ifadenin bir parçasıdır. Ama C# dilindeki bildirimdeki yıldız karakteri referanstan çıkarma işleci (**de-reference operator**) (*) olup veri tipinin bir parçasıdır.

Aşağıdaki örnekte verilen kod C# dilinde iki `int` göstericisi bildirmesine rağmen C ve C++ dillerinde ilki gösterici, ikincisi değişken olarak bildirilir.

```
int* a,b;
/*
```

```
C ve C++ olarak eşdeğeri: int* a; int b;
C# eşdeğeri: int* a; int* b;
*/
int *bir, *iki; //Hata:C# dilinde derlenmez.
/*
Ancak C ve C++ dillerinde geçerli iki farklı göstericidir.
*/
```

§13.7.4 Üyelere Erişim

C ve C++ gibi C# dilinde de bir yapı ya da sınıf örneğinin üyelerine bir gösterici aracılığıyla erişme, **ok işleci (arrow operator) ->** ile yapılır.

```
unsafe
{
    Koordinat v;
    v.X = 5;
    v.Y = 10;
    Koordinat* ptr = &v;
    int x = ptr->X;
    int y = ptr->Y;
    string s = ptr->ToString();
    Console.WriteLine(x); // 5
    Console.WriteLine(y); // 10
    Console.WriteLine(s); // Koordinat
}
struct Koordinat
{
    public int X;
    public int Y;
}
```

13.8 Düşün ve Çöz

- ◊ **Düşün:** Örtük (**implicit**) ve açık (**explicit**) tip dönüşümü arasındaki fark nedir? Örtük dönüşümün güvenli kabul edilmesinin nedeni nedir? Veri kaybına yol açabilecek bir dönüşümün açık yapılmasını zorunlu kılan tasarım kararı nedir?
- ◊ **Düşün:** is ve as işleçleri arasındaki fark nedir? Bir taban sınıf referansının türemiş sınıf tipine dönüşürülmesinde **InvalidCastException** riskini nasıl önlersiniz? C#7 ile gelen desen eşleştirme (**pattern matching**) bu süreçte nasıl kolaylaştırır?
- ◊ **Düşün:** Kutulama (**boxing**) ve kutudan çıkarma (**unboxing**) işlemleri neden performans maliyeti yaratır? Sıkça tekrarlanan işlemlerde kutulama kaçınılmaz hale geldiğinde neler yapılabilir? Jenerik yapılar bu sorunu nasıl çözer?
- ◊ **Düşün:** C#'ta unsafe kod bloğu ve göstericiler (**pointer**) ne zaman kullanılır? **Managed** ve **unmanaged** bellek arasındaki fark nedir? .NET'te bellek güvenliğini sağlayan mekanizmalar nelerdir?
- ◊ **Düşün:** **Span<T>** ve **Memory<T>** tipleri göstericilere alternatif olarak sunulmuştur. Bu tiplerin güvenli bellekle çalışma avantajı nedir? Büyük dizilerin bir bölümü üzerinde kopyasız (**zero-copy**) işlem yapmanın önemi nedir?
- ◊ **Düşün:** Sabitlenmiş ifade (**fixed statement**) neden gereklidir? Çöp toplayıcının belleği taşıması (**GC compaction**) gösterici aritmetiğini nasıl tehlikeli hale getirir? **fixed** bu riski nasıl ortadan kaldırır?
- ◊ **Düşün:** Tip dönüşümü operatörleri (**implicit/explicit operator**) kendi sınıflarınıza nasıl eklenir? Bu mekanizmanın aşırı kullanımı kodun okunabilirliğini nasıl olumsuz etkiler?
- ◊ **Çöz:** Bir Sıcaklık sınıfı tanımlayınız: **implicit operator** ile **double**'a, **explicit operator** ile **int**'e dönüşüm sağlasın. Celsius ve Fahrenheit arası çevirimi implicit operatörle uygulayın ve test edin.
- ◊ **Çöz:** Hayvan taban sınıfından türeyen Köpek ve Kedi sınıflarınız olsun. is, as ve C#7 desen eşleştirme söz dizimini kullanarak farklı hayvan tiplerini ele alan bir yöntemi üç farklı şekilde yazınız ve karşılaştırınız.
- ◊ **Çöz:** Kutulama maliyetini ölçmek için: 1 milyon eleman üzerinde **int\object** kutulaması yapan ve jenerik **List<int>** kullanan iki döngünün çalışma süresini Stopwatch ile ölçüp karşılaştıran bir program yazınız.
- ◊ **Çöz:** **Span<T>** kullanarak bir büyük dizinin ortasındaki 100 elemanlık bölümü kopyalamadan işleyen (toplamı bulan) bir program yazınız. Aynı işlemi normal dizi kopyalamayla yapan versiyonla karşılaştırın.

- ◇ **Çöz:** `unsafe` bloğu içinde gösterici aritmetiği kullanarak bir `byte` dizisini tersine çeviren (`reverse`) bir yöntem yazınız. Ardından aynı işlemi `Span<T>.Reverse()` ile yapın ve sonuçları karşılaştırın.